COSC204 Computer Systems

Lecture 22: Virtual Memory

Zhiyi Huang

School of Computing, University of Otago

Overview

Last lecture: memory management

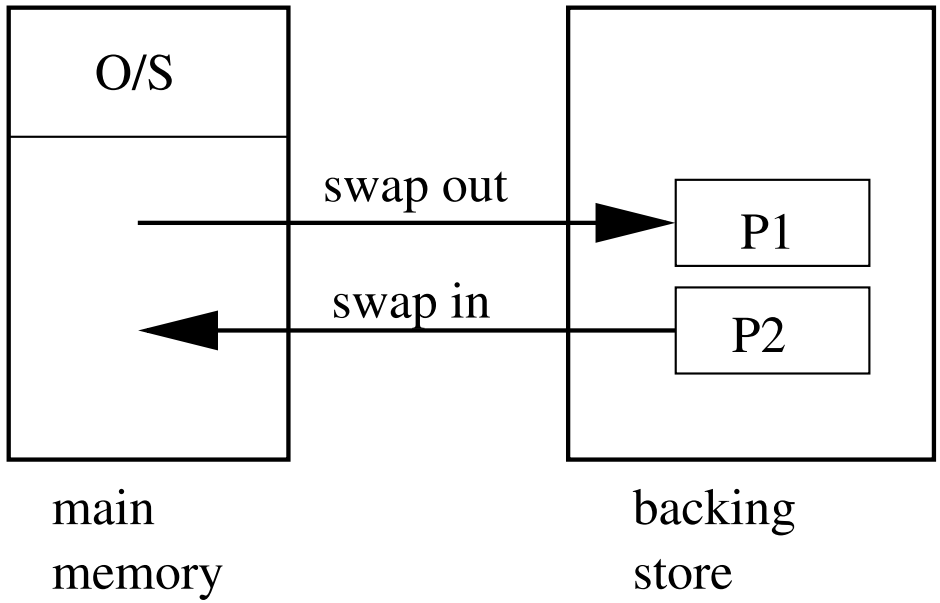
- Storage hierarchy
- Process memory image
- Logical and physical addresses
- Paging

This lecture: more advanced memory management

- Swapping and virtual memory
- Demand paging
- Page replacement algorithms
- Frame allocation
- Thrashing

Swapping: moving processes between memory & disk

If there is not enough physical memory, we need to swap processes out of the main memory to the secondary storage (e.g., disk).



- Swapping allows more processes to multitask at any given time.
- When a process is ready, it is swapped into the main memory.

Processes may only be partially loaded into memory

Processes/programs are often not fully in memory.

- Dynamic loading: load a code/routine/function when it is called. Some code may not be needed most of the time, e.g., code for handling rare error conditions.

Advantages:

- A process will not be directly constrained by the amount of available physical memory.
- It should be possible to increase the number of processes that are multitasking.
- During multitasking, swapping in and out of memory will be faster than if moving whole processes in and out of physical memory.

Virtual memory: extend memory to secondary storage

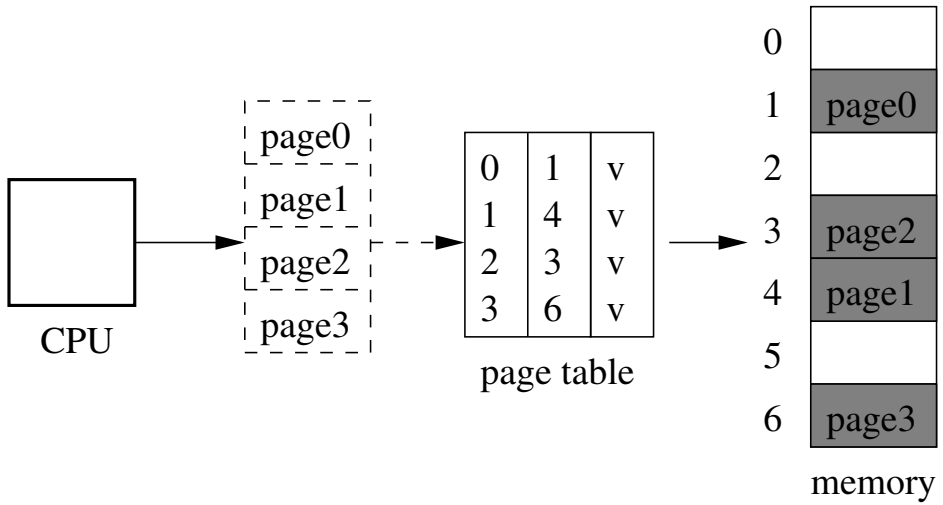
The idea that processes do not need to be fully in memory in order to run is implemented in the virtual memory system.

A virtual memory system can extend the main memory (e.g., RAM) to the secondary storage (e.g., disk) and provide the means to *dynamically load processes' memory while they execute*.

- The programmer deals with a logical memory space (AKA virtual memory), just as in a paging memory scheme.
- The memory manager in OS kernel is not only in charge of mapping logical addresses to physical addresses, but *also* of actually loading pages of the process into the main memory from the secondary storage (file system or swap space).

Demand paging—mapping without loading data

The paging system we saw last lecture was introduced to eliminate external fragmentation.



- Demand paging: do not load a page into main memory until it is referenced by the CPU (i.e., load pages on demand).

Implementing demand paging

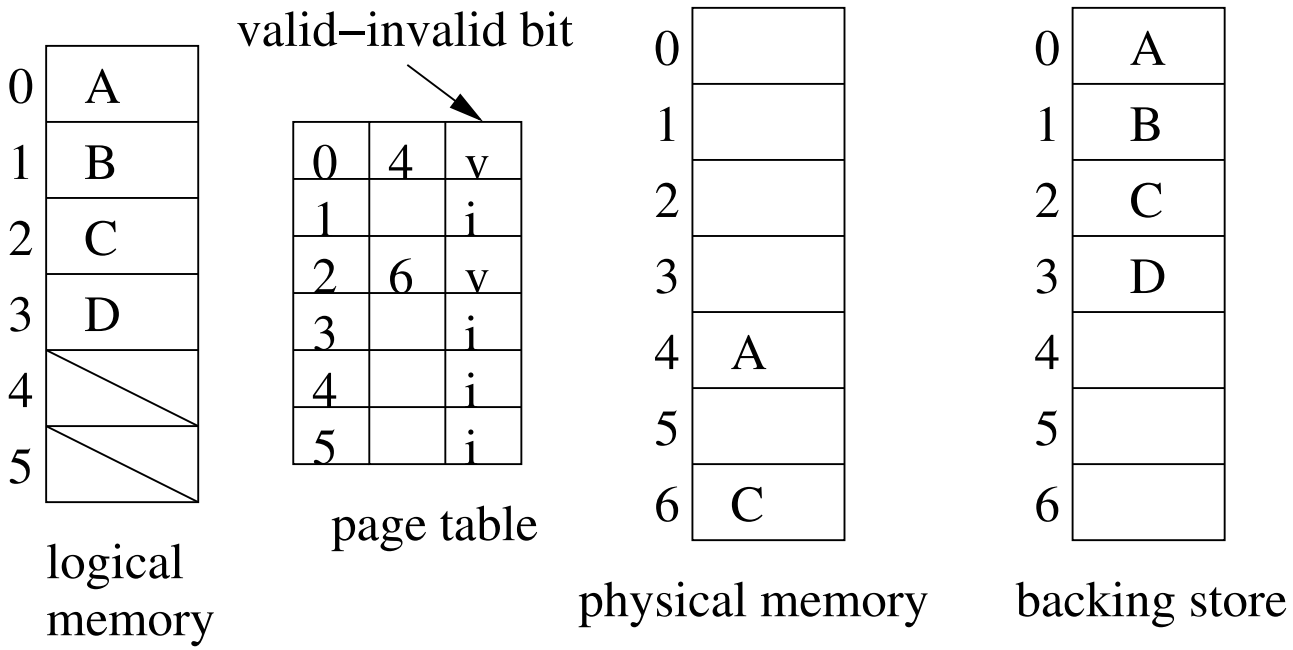
In a standard paging scheme, an entry in the page table typically contains a number of additional control bits:

- Valid/invalid bit: used to indicate whether the page number referred to by a process is inside its allocated number of pages.
- Protection bit: e.g., whether this page is read-write or read-only.

In demand paging, the memory manager needs some way of knowing whether a given page in a process has been loaded or not.

The valid/invalid bit is often used to carry this information.

Depicting an example of demand paging



Two important points to note:

- There are now two different uses for the invalid bit.
- Two copies of data are stored, i.e., A is in RAM *and* on disk. These copies must be kept consistent!

Page faults allow the OS to implement demand paging

Accessing an invalid page in the table results in a trap (page fault) to the operating system.

What the operating system does:

- Determine, using the memory mapping information in the PCB, if the page fault is due to a valid memory access but the page frame is not available, or if it is due to an invalid memory access.
- If it is due to an invalid memory access, terminate the process.
- Otherwise:
  - Find a free frame in main memory.
  - Read the page from the disk into the free frame.
  - Modify the page table appropriately.
  - Resume the process at the point the page fault occurred.

How to know if a memory access is valid? System calls like `mmap()` set up legal virtual memory areas (VMAs) that are checked against.

Page replacement algorithms

If main memory is full when a page fault is generated, one of the pages currently being held in memory needs to be replaced.

Some extra steps are added into the OS page-servicing routine:

- Find the desired page on the backing store (secondary storage).
- Find a free frame of memory.
  - If there is a free frame in memory, use it. Otherwise:
  - Select a frame to swap out.
  - Save it to the backing store.
  - Proceed as before.

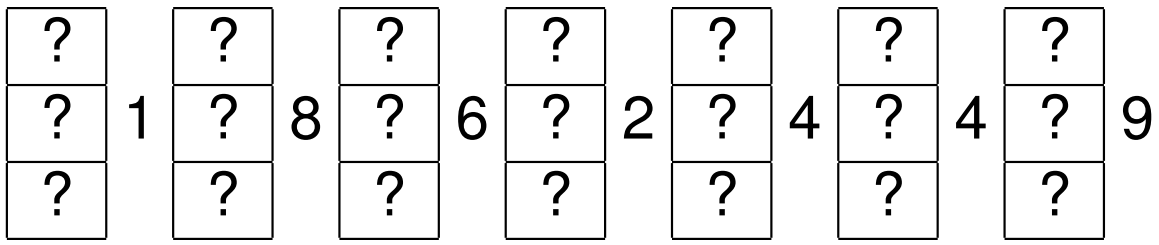
Page replacement: which page to swap out?

There are various methods for determining which page to swap out.

These are often evaluated with respect to a sequence of references ("reference string") to pages requested.

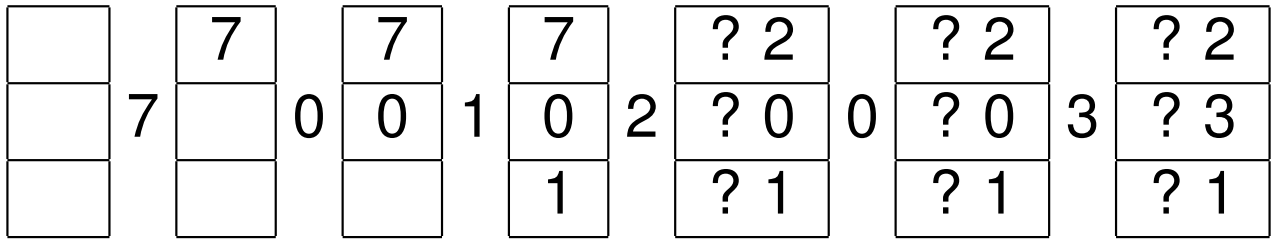
The contents of main memory can be examined after each request.

An example for a memory containing 3 frames, and the pages being inserted in the order 1, 8, 6, 2, 4, 4, 9:



FIFO page replacement algorithm

Basically, FIFO replaces the oldest page in memory. An example:

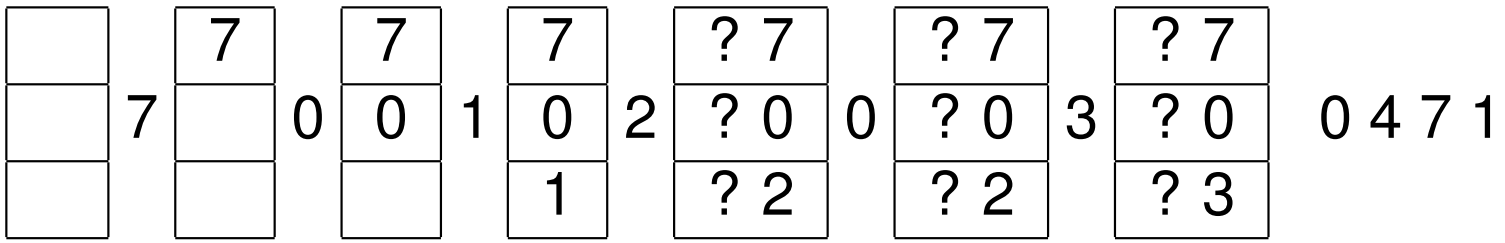


- Advantages: simple to understand/implement.
- Disadvantages: the first page to be loaded might actually be highly used, but this is not taken into account.

Also BELADY'S ANOMALY: it's possible to increase the number of page faults when increasing the number of frames in memory. (NB: you would normally expect the page fault rate to fall as the number of frames increases.)

Optimal page replacement algorithm

Optimal: replace page that will not be used until furthest into the future (i.e., within given sequence of page accesses). For example:



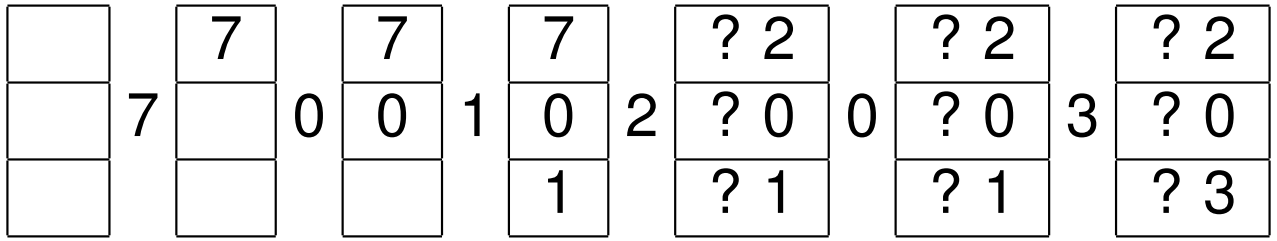
NB: revert to FIFO for pages that will not be used again.

Advantages / disadvantages of the Optimal algorithm?

- Advantage is in the name: the algorithm is optimal!
- Disadvantages are similar to SJF scheduling: you don't typically actually know the future sequence of references.

Least recently used (LRU) page replacement

In LRU ('least recently used') we replace the page that has not been used for the longest period of time. For example:



Advantages / disadvantages of LRU?

- Advantage: it is optimal if you look backward in time.
- Disadvantage: it is time-consuming to keep a record of LRU.

We are making a key assumption regarding optimality: That future memory references resemble past ones.

Frame allocation algorithms

It makes sense that processes should be allocated a minimum number of frames.

For any given CPU type there is a minimum number of frames that *must* be allocated to a process, determined by the maximum number of memory locations that can be accessed in a single instruction.

How can we best distribute frames among *n* processes, in a multitasking scenario?

- Equal allocation
- Proportional allocation
- Local versus global replacement schemes

Thrashing—a virtual memory situation to avoid

If a process has not been allocated sufficient frames, it thrashes:

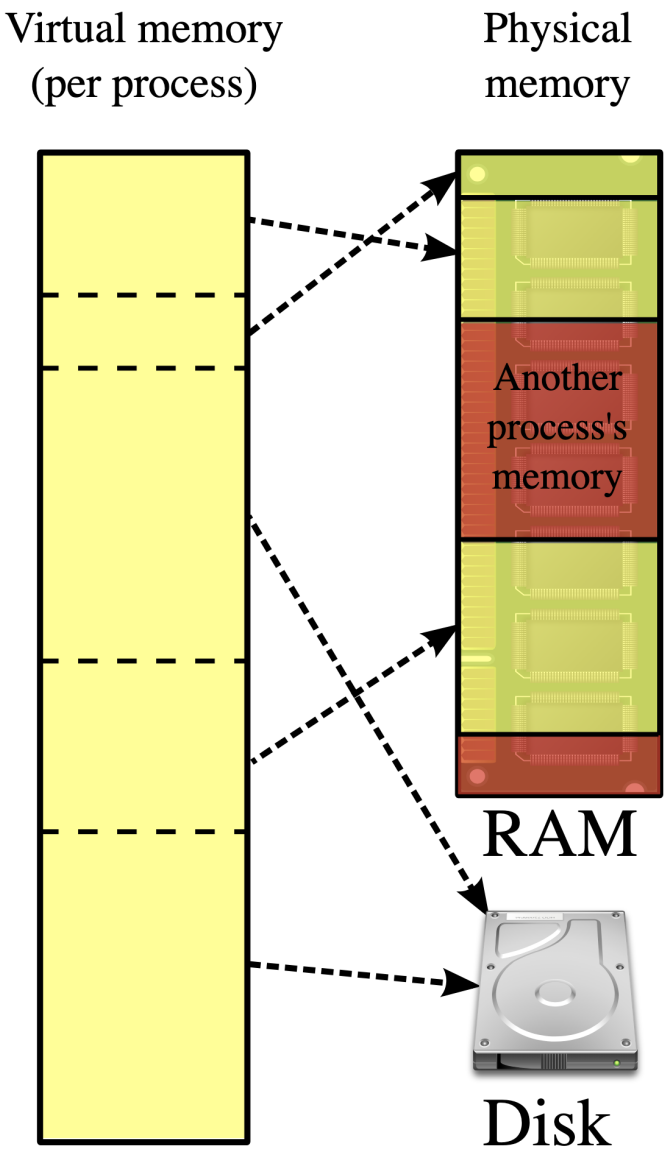
- A page fault occurs, requiring a page to be replaced.
- The replaced page is itself needed shortly after.

A process is thrashing if it spends more time paging than executing.

To avoid thrashing, we need to find the size of the working set of a process, and allocate that size of memory to the process.

A working set of a process is the number of pages a process needs in order to execute for reasonable amounts of time without page faults being caused.

Thrashing due to oversized active virtual memory



Reading

For this lecture, you should read Chapter 9 (Sections 1–7) of Silberschatz *et al.*